

Object-Oriented Programming Overview

3

Fundamental CONCEPTS of OOP

- Objects & Classes
- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

Objects

4

- An object is:
 - A set of related data which identifies the current *state* of the object.
 - + a set of behaviors

Example objects:

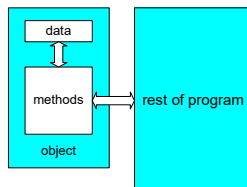
human entities	physical objects	mathematical entities
employees	cars in a traffic-flow simulation	points in a coordinate system
customers	aircraft in an air-traffic control system	complex numbers
students	electrical components in a circuit-design program	time

- Car object in a traffic-flow simulation:
 - data = ?
 - methods = ?

Object-Oriented Programming Overview

5

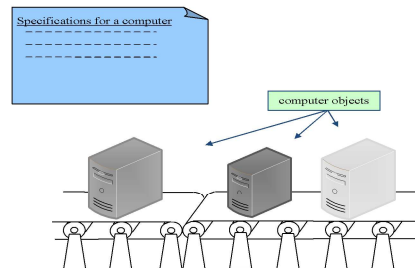
- Benefits of OOP:
 - Programs are more understandable -
 - Fewer errors -



Objects and Classes- OOP Overview

6

- A *class* is a description for a set of objects.



Objects and Classes –OOP Overview

8

- A *class* is a description for a set of objects. The description consists of:
 - a list of variables
 - + a list of methods
- Classes can define two types of variables – instance variables and class variables.
- Classes can define two types of methods – instance methods and class methods.

Objects and Classes-OOP Overview

9

- A class's *instance variables* specify the type of data that an object can store.
 - For example, class for computer objects, and the class contains a `hardDiskSize` instance variable.
- A class's *instance methods* specify the behavior that an object can exhibit.
 - For example, if you have a class for computer objects, and the `Computer` class contains a `printSpecifications` instance method, then each computer object can print a specifications report.

Object-Oriented Programming Overview

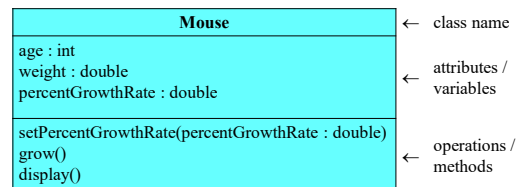
10

- The term "instance" in "instance variable" and "instance method" indicate that the instance variables and instance methods are associated with a particular object instance.
- Class variables and class methods are associated with an entire class. For example, the `Math` class contains the `PI` class variable and the `round` class method.

UML Class Diagram

11

- UML:
- Example:
 - UML class diagram for a `Mouse` class:



First OOP Class

12

```

/*****
 * Mouse.java
 * Dean & Dean
 *
 * This class models a mouse for a growth simulation program.
 *****/

public class Mouse
{
    private int age = 0;           // age of mouse in days
    private double weight = 1.0;  // weight of mouse in grams
    private double percentGrowthRate; // % weight increase per day

    //*****

    // This method assigns the mouse's percent growth rate.
    public void setPercentGrowthRate(double percentGrowthRate)
    {
        this.percentGrowthRate = percentGrowthRate;
    } // end setPercentGrowthRate
}

```

instance variable declarations

parameter

method body

To access instance variables, use this dot.

First OOP Class

14

```

/*****
 *
 * This method simulates one day of growth for the mouse.
 */

public void grow()
{
    this.weight += (.01 * this.percentGrowthRate * this.weight);
    this.age++;
} // end grow

/*****
 *
 * This method prints the mouse's age and weight.
 */

public void display()
{
    System.out.printf(
        "Age = %d, weight = %.3f\n", this.age, this.weight);
} // end display
} // end class Mouse

```

private and public Access

15

- Private *access modifiers*
If you declare a member to be `private`, then the member can be accessed only from within the member's class.
- public *access modifiers*
If you declare a member to be `public`, then the member can be accessed from anywhere

Driver Class

16

```

/*****
 * MouseDriver.java
 * Dean & Dean
 *
 * This is a driver for the Mouse class.
 *****/

import java.util.Scanner;

public class MouseDriver
{
    public static void main(String[] args)
    {
        Scanner stdIn = new Scanner(System.in);
        double growthRate;
        Mouse gus = new Mouse();
        Mouse jaq = new Mouse();
    }
}

```

Reference Variables and Instantiation

18

- To declare a *reference variable* (which holds the address in memory where an object is stored):
`<class-name> <reference-variable>;`
- To instantiate/create an object and assign its address into a reference variable:
`<reference-variable> = new <class-name> ();`
- Example code:

```
Mouse gus;  
gus = new Mouse();
```

Labels: **declaration** (points to `Mouse gus;`), **instantiation** (points to `gus = new Mouse();`)
- This single line is equivalent to the above two lines:

```
Mouse gus = new Mouse();
```

Label: **initialization** (points to `Mouse gus = new Mouse();`)

Calling a Method

19

- Call/invoke an instance method using this syntax:
`<reference-variable>.<method-name> (<comma-separated-arguments>);`
- Three example instance method calls from the `MouseDriver` class:

```
gus.setPercentGrowthRate(growthRate);  
gus.grow();  
gus.display();
```

Calling Object

20

- A *calling object* is the object that appears at the left of the dot in a call to an instance method.
- Can you find the calling objects below?

```
public static void main(String[] args)  
{  
    Scanner stdIn = new Scanner(System.in);  
    double growthRate;  
    Mouse gus = new Mouse();  
  
    System.out.print("Enter growth rate as a percentage: ");  
    growthRate = stdIn.nextDouble();  
    gus.setPercentGrowthRate(growthRate);  
    gus.grow();  
    gus.display();  
} // end main
```

The `this` Reference

21

- The *this* reference:
 - When used in conjunction with a dot and an instance variable, "this" is referred to as the *this reference*.

Example from the `Mouse` class's `grow` method:

```
this.weight += (.01 *  
    this.percentGrowthRate * this.weight);
```

- The *this* reference is used inside of a method to refer to the object that called the method;
So, what's so great about having a special name for the calling object inside of a method? Why not just use the original name, `gus` or `jaq`, inside the method?

Default Values

22

- A variable's *default value* is the value that the variable gets if there's no explicit initialization.
- Mouse class's instance variable declarations:


```
private int age = 0;
private double weight = 1.0;
private double percentGrowthRate;
```

 - explicit initializations (points to age and weight)
 - percentGrowthRate gets default value of 0.0
- Here are the default values for instance variables:
 - integer types get 0
 - floating point types get 0.0
 - boolean types get false
 - reference types get null
 - Note that a String is a reference type so it gets null by default.

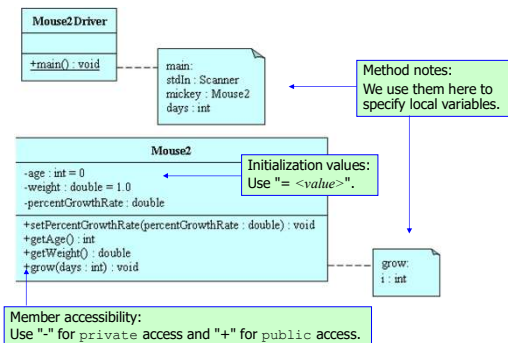
Variable Persistence

23

- A variable's *persistence* is how long a variable's value survives before it's wiped out.
- Instance variables persist for the duration of a particular object.

UML Class Diagram for Next Version of the Mouse Program

32



Local Variables

33

- A *local variable* is a variable that's declared inside a method. That's different from an instance variable, which is declared at the top of a class, outside all the methods.
- A local variable is called "local" because it can be used only inside of the method in which it is declared – it is completely local to the method.

Mouse2Driver Class

34

```
import java.util.Scanner;

public class Mouse2Driver
{
    public static void main(String[] args)
    {
        Scanner stdin = new Scanner(System.in);
        Mouse2 mickey = new Mouse2();
        int days;
        mickey.setPercentGrowthRate(10);
        System.out.print("Enter number of days to grow: ");
        days = stdin.nextInt();
        mickey.grow(days);
        System.out.printf("Age = %d, weight = %.3f\n",
            mickey.getAge(), mickey.getWeight());
    } // end main
} // end class Mouse2Driver
```

local variables

Mouse2 Class

35

```
import java.util.Scanner;
public class Mouse2
{
    private int age = 0;
    // age in days
    private double weight = 1.0; // weight in grams
    private double percentGrowthRate; // % daily weight gain

    //*****
    public double getWeight()
    {
        return this.weight;
    } // end getWeight

    //*****
    public void grow(int days)
    {
        for (int i=0; i<days; i++)
        {
            this.weight += (.01 * this.percentGrowthRate * this.weight);
            this.age += days;
        } // end grow
    } // end class Mouse2

    //*****
    public void setPercentGrowthRate(double percentGrowthRate)
    {
        this.percentGrowthRate = percentGrowthRate;
    } // end setPercentGrowthRate

    //*****
    public int getAge()
    {
        return this.age;
    } // end getAge
}
```

return Statement

37

- The `return` statement allows you to pass a value from a method back to the place where the method was called. Note the following example.

- From the `Mouse2` class:

```
public int getAge()
{
    return this.age;
} // end getAge
```

return type

return statement

- From the `Mouse2Driver` class:

```
System.out.printf("Age = %d, weight = %.3f\n",
    mickey.getAge(), mickey.getWeight());
```

method call

void Return Type

38

- As shown in the below `grow` method from the `Mouse2` class, if a method does not return a value, then the method must specify `void` for its return type.

```
public void grow(int days)
{
    for (int i=0; i<days; i++)
    {
        this.weight += (0.01 * this.percentGrowthRate * this.weight);
    }
    this.age += days;
} // end grow
```

Empty return Statement

39

- For methods with a `void` return type, it's legal to have an empty `return` statement. The empty `return` statement looks like this:

```
return;
```

Empty return Statement

40

- Suppose you'd like to model mouse growth only up through mouse adolescence. This `grow` method does that by stopping a mouse's growth after 100 days:

```
public void grow(int days)
{
    int endAge;
    endAge = this.age + days;
    while (this.age < endAge)
    {
        if (this.age >= 100)
        {
            return; ← empty return statement
        }
        this.weight +=
            .01 * this.percentGrowthRate * this.weight;
        this.age++;
    } // end while
} // end grow
```

Empty return Statement

41

- Code that uses an empty `return` statement(s) can always be replaced by code that does not use the empty `return` statement(s). For example, here's a return-less version of the previous `grow` method:

```
public void grow(int days)
{
    int endAge;
    endAge = this.age + days;
    if (endAge > 100)
    {
        endAge = 100;
    }
    while (this.age < endAge)
    {
        this.weight +=
            (.01 * this.percentGrowthRate * this.weight);
        this.age++;
    } // end while
} // end grow
```

Local Variable Default Values

43

- The default value for a local variable is *garbage*.
- If a program attempts to access a variable that contains garbage, the compiler generates an error. For example, what happens when the following method is compiled?

```
public void grow(int days)
{
    for (int i; i < days; i++)
    {
        this.weight +=
            (.01 * this.percentGrowthRate * this.weight);
    }
    this.age += days;
} // end grow
```

- Since `i` is no longer assigned zero, `i` contains garbage when the `i < days` condition is tested. That causes the compiler to generate this error message:
variable `i` might not have been initialized

Argument Passing

45

- What is the output for the following `Mouse3Driver` and `Mouse3` classes?

```
public class Mouse3Driver
{
    public static void main(String[] args)
    {
        Mouse3 minnie = new Mouse3();
        int days = 365;
        minnie.grow(days);
        System.out.println("# of days aged = " + days);
    } // end main
} // end class Mouse3Driver
```

Argument Passing

46

```
public class Mouse3
{
    private int age = 0;           // age in days
    private double weight = 1.0;  // weight in grams
    private double percentGrowthRate = 10; // % daily weight gain

    //*****

    public void grow(int days)
    {
        this.age += days;
        while (days > 0)
        {
            this.weight +=
                (.01 * this.percentGrowthRate * this.weight);
            days--;
        }
    } // end grow
} // end class Mouse3
```

Specialized Methods

49

- Accessor methods -**

- They simply get/access the value of an instance variable.
- Example:

```
public int getAge()
{
    return this.age;
}
```

- Mutator methods -**

- They simply set/mutate the value of an instance variable.
- Example:

```
public void setPercentGrowthRate(double percentGrowthRate)
{
    this.percentGrowthRate = percentGrowthRate;
} // end setPercentGrowthRate
```

Specialized Methods

50

- boolean methods -**

- They check the truth or falsity of some condition.
- They always return a `boolean` value.
- They should normally start with "is".

- For example, here's an `isAdolescent` method that determines whether a `Mouse` object's age is ≤ 100 :

```
public boolean isAdolescent()
{
    if (this.age <= 100)
    {
        return true;
    }
    else
    {
        return false;
    }
} // end isAdolescent
```

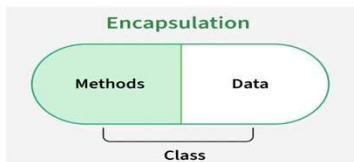
- Here's how the `isAdolescent` method might be used in `main`:

```
Mouse pinky = new Mouse();
...
if (pinky.isAdolescent() == false)
{
    System.out.println(
        "The Mouse's growth is no longer" +
        " being simulated - too old.");
}
```


Encapsulation

33

- Encapsulation - a mechanism of wrapping the data *variables* and code acting on the data *methods* together as single unit.



Encapsulation

34

- How do we achieve encapsulation?
- Declare class variables/attributes as **private**
- Provide public **get** and **set** methods to access and update the value of a **private** variable
- Apply proper access modifiers:
- Why Encapsulation?

Code to illustrate encapsulation

35

```
class Encaps {
    private String name;
    public String getName() { return name; } // Getter method used to get the data
    public void setName(String name) // Setter method is used to set or modify the data
    {
        this.name = name;
    }
}

public class Check {
    public static void main(String[] args){
        Encaps p = new Encaps();
        p.setName("IGovender");
        System.out.println("Name=> " + p.getName());
    }
}
```

Inheritance

36

- It is the mechanism in Java by which one class is allowed to inherit the features (fields and methods) of another class. We are achieving inheritance by using **extends** keyword. Inheritance is also known as "**is-a**" relation.
- Superclass also known as parent class
- Subclass also known as child class
- Reusability of fields and methods

Polymorphism

37

- Polymorphism means "many forms", and it occurs when we have many classes that are related to each other by inheritance

Abstraction

38

- Only the essential details are displayed to the user

- Example: a real-life example of a man driving a car.

Driver knows that pressing the accelerator will increase the speed or applying brakes will slow/stop the car.

He does not know the inner mechanism of how the brakes and accelerator works.

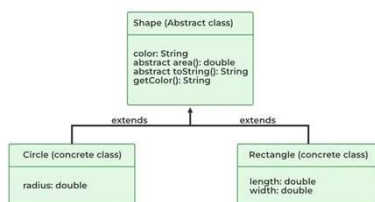
Abstraction is achieved by:
Interfaces and abstract classes

Abstract class

39

Characteristics

- Cannot be instantiated
- May contain abstract methods (without body) and Concrete methods (with body)



```

return Math.PI * Math.pow(radius, 2);
}
@Override public String toString(){
    return "Circle color is " + super.getColor()
    + "and area is : " + area();
}
}

class Rectangle extends Shape{
    double length;
    double width;

    public Rectangle(String color, double length, double width)
    {
        // calling Shape constructor
        super(color);
        System.out.println("Rectangle constructor called");
        this.length = length;
        this.width = width;
    }

    @Override double area() { return length * width; }
    @Override public String toString()
    {
        return "Rectangle color is " + super.getColor()
        + "and area is : " + area();
    }
}

public class Test {
    public static void main(String[] args)
    {
        Shape s1 = new Circle("Red", 2.2);
        Shape s2 = new Rectangle("Yellow", 2, 4);
        System.out.println(s1.toString());
        System.out.println(s2.toString());
    }
}

abstract class Shape{
    String color;

    // these are abstract methods
    abstract double area();
    public abstract String toString();

    // abstract class can have the
    constructor
    public Shape(String color){
        System.out.println("Shape
    constructor called");
        this.color = color;
    }

    // this is a concrete method
    public String getColor(){
        return color;
    }
}

class Circle extends Shape{
    double radius;
    public Circle(String color, double
    radius){
        // calling Shape constructor
        super(color);
        System.out.println("Circle
    constructor called");
        this.radius = radius;
    }
}
  
```

40



Interface

41

- Interface is a blueprint of a class
- Achieves 100% abstraction
- Contains abstract methods and constants –but no method bodies
- Keyword for implementation- “implements”



Advantages and Disadvantages of Abstraction

42

- Advantages
 - makes complex systems easier to understand
 - keeps different part of the system separated.
 - maintains code more efficiently.
 - increases the security
- Disadvantages
 - unnecessary complexity if overused.
 - May reduce flexibility
 - debugging harder for unfamiliar users.
 - abstraction layers can affect performance.
- Common Mistakes to Avoid
 - Not Implementing Abstract Methods: .
 - Overusing Abstraction:
 - Inconsistent Method Signatures in Subclasses: